

CivicCare - Online Complaint Registration System.

INTRODUCTION:

Welcome to **CivicCare**—a secure and user-friendly civic complaint management system designed to make reporting and tracking public issues easier and more transparent.

Citizens can register, securely log in, submit complaints with details such as category, location, priority, and description, and track their complaint status.

CivicCare provides separate dashboards for **Citizens, Agents, and Administrators**. Administrators can manage complaints, create agents, and assign complaints, while agents can handle assigned complaints and update their status.

With its simple interface and role-based access, CivicCare improves communication, complaint management, and transparency between citizens and civic authorities.

SCENARIO Based Case Study:

Background

“**CivicCare**” is a web-based civic complaint management system designed to make reporting and tracking public issues simple, secure, and efficient. The system allows citizens to register complaints related to roads, water, electricity, garbage, street lights, and other civic issues. It provides role-based dashboards for citizens, agents, and administrators to improve complaint management and transparency.

Scenario

User Persona:

Name:

Aarav Sharma

Age: 28

Profession: Software Engineer

Goal: Report civic issues in his locality and easily track the progress and resolution of his complaints.

Challenges:

Busy work schedule, difficulty reporting complaints through traditional methods, lack of

transparency in complaint status, and difficulty knowing whether the complaint has been assigned or resolved.

The Challenge

Aarav had faced difficulties reporting civic issues through traditional methods and found it difficult to track whether his complaints were being handled. He needed a system that could:

- Allow quick and easy complaint registration
- Provide clear complaint status tracking
- Enable proper assignment and management of complaints
- Provide secure access for citizens, agents, and administrators

Solution: How CivicCare Helped

When Aarav used **CivicCare**, he could securely register and log in to the system and submit his civic complaint with details such as category, location, priority, and description. The complaint was then managed through the role-based system.

- **Complaint Registration:** Citizens can submit civic complaints with complete issue details.
- **Complaint Tracking:** Citizens can monitor the current status of their complaints.
- **Admin & Agent Management:** Administrators can assign complaints to agents, while agents can update the complaint status.

Within the system, Aarav was able to submit his complaint, track its progress, and receive a clear status update until resolution, making the complaint management process more transparent and efficient.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the **SpeakEase** Web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable agricultural platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing.
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries.
- **React.js:** A JavaScript library for building dynamic and responsive user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js:** A lightweight web framework for building RESTful APIs.
- **MongoDB:** A NoSQL database used to store structured and unstructured data related to farms, users, and resources.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

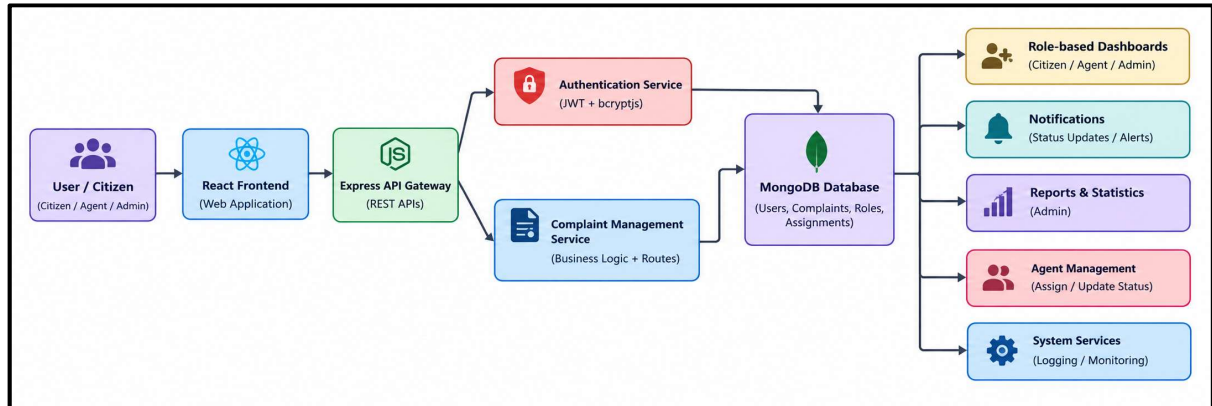
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.

- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

TECHNICAL ARCHITECTURE:



Technical Architecture Overview

Our **CivicCare** application is built on a secure and scalable architecture designed to provide an efficient platform for registering, managing, assigning, and tracking civic complaints.

User Interface (UI):

A clean and user-friendly React interface enables **citizens, agents, and administrators** to access their respective dashboards, submit complaints, manage complaints, and track status.

Express API Gateway:

Acts as the central entry point for frontend requests and handles communication between the React frontend and backend services through REST APIs.

Authentication Service:

Secures user access through **JWT authentication and bcryptjs password hashing**, providing role-based access for citizens, agents, and administrators.

Complaint Management Service:

Manages complaint registration, complaint details, categories, locations, priorities, and status updates. It also supports complaint assignment to agents.

Database:

MongoDB stores user profiles, roles, complaint details, priorities, statuses, and agent assignment information.

Role-Based Dashboards:

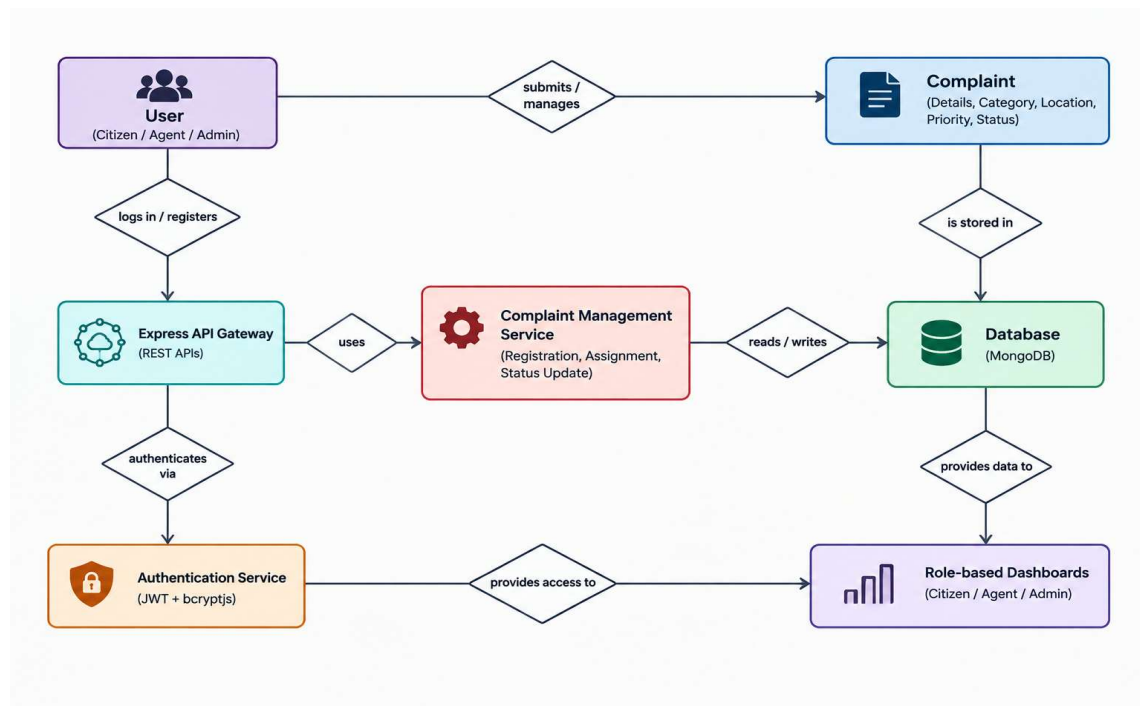
Provides separate dashboards for **Citizens, Agents, and Administrators**. Citizens track complaints, agents handle assigned complaints, and administrators manage and assign complaints.

Agent Management:

Allows administrators to create agent accounts and assign complaints to agents. Agents can view assigned complaints and update their status.

Together, these components create a **secure, organized, and efficient civic complaint management system** that improves complaint tracking, assignment, and transparency.

ER DIAGRAM:



Entities and Relationships

1. User

- Represents the **citizens, agents, and administrators** using the CivicCare system.
- Each user has a unique profile with details such as **name, email, password, and role**.
- **Relationships:**

- **Submits → Complaint:** Citizens can submit civic complaints with details such as category, location, priority, and description.
- **Tracks → Complaint:** Citizens can view and track the status of their submitted complaints.
- **Manages → Complaint:** Administrators and agents manage complaints according to their assigned roles.

2. Complaint

- Represents a civic issue reported by a citizen.
- Stores details such as **title, description, category, location, priority, status, and assigned agent**.
- **Relationships:**
 - **Belongs to → User:** Each complaint is associated with the citizen who submitted it.
 - **Assigned to → Agent:** Administrators can assign complaints to agents for handling.
 - **Updates → Status:** Agents and administrators can update the complaint status.

3. Agent

- Represents the users responsible for handling and resolving assigned civic complaints.
- Agents can view complaints assigned to them and manage their status.
- **Relationships:**
 - **Receives → Complaint:** Agents receive complaints assigned by administrators.
 - **Updates → Complaint:** Agents update the status of assigned complaints.

4. Admin

- Represents the administrator responsible for managing the CivicCare system.
- Admins can view complaints, create agents, assign complaints, and manage complaint information.
- **Relationships:**
 - **Manages → Complaint:** Admins manage and monitor all submitted complaints.
 - **Assigns → Agent:** Admins assign complaints to appropriate agents for resolution.
 - **Creates → Agent:** Admins can create agent accounts for complaint handling.

Key Features:

1. User Management

- **User Registration and Authentication:**

Allows citizens, agents, and administrators to register/login securely and access features according to their assigned roles.

2. Complaint Management

- **Complaint Registration:**

Allows citizens to submit civic complaints with details such as title, description, category, location, and priority.

- **Complaint Assignment:**

Allows administrators to assign complaints to agents for proper handling and resolution.

3. Complaint Tracking and Resolution

- **Status Tracking:**

Allows citizens to track complaint status, while agents and administrators can update statuses such as **Pending, In Progress, Resolved, and Rejected.**

ROLES AND RESPONSIBILITIES:

User:-

Registration: Users can create an account on **CivicCare** by providing necessary details such as name, email, and password.

Profile Management: Users can access their account information and manage their registered details.

Complaint Registration: Citizens can submit civic complaints by providing details such as title, description, category, location, and priority.

Complaint Tracking: Citizens can view their submitted complaints and track their current status.

Role-Based Access: Users can access features according to their assigned role as **Citizen, Agent, or Admin.**

Logout: Finally, users can securely logout from the **CivicCare** system.

Admin:-

User Management: Admins can manage user accounts and maintain access according to their assigned roles.

Complaint Management: Admins can view, manage, and monitor all civic complaints submitted by citizens.

Agent Management: Admins can create agent accounts and assign complaints to suitable agents for handling.

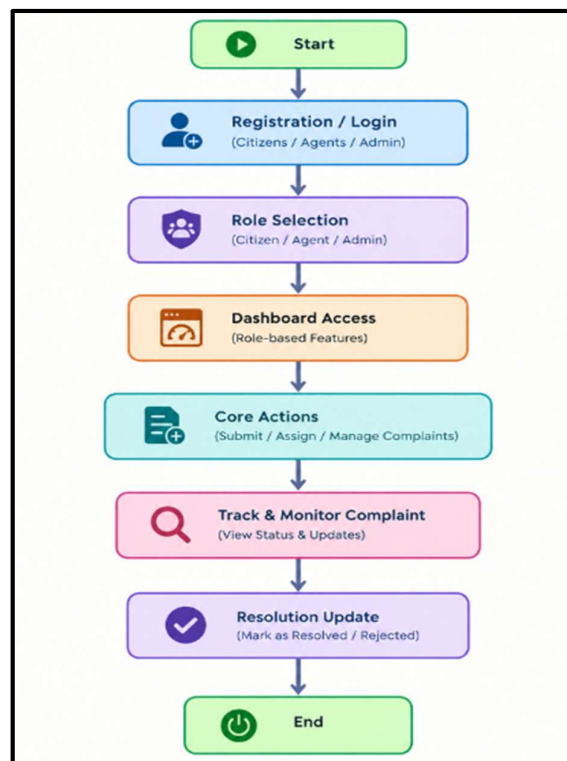
Complaint Assignment: Admins can assign complaints to agents and manage complaint priorities.

Status Monitoring: Admins can monitor complaint statuses such as **Pending, In Progress, Resolved, and Rejected**.

Complaint Updates: Admins can update complaint information and ensure that complaints are properly managed throughout the resolution process.

Logout: Admins can securely logout from the **CivicCare** system

User Flow:



Start: The user opens the **CivicCare** application.

Registration / Login: The user registers or logs in using their email and password.

Dashboard Access: The user accesses the dashboard according to their role — Citizen, Agent, or Admin.

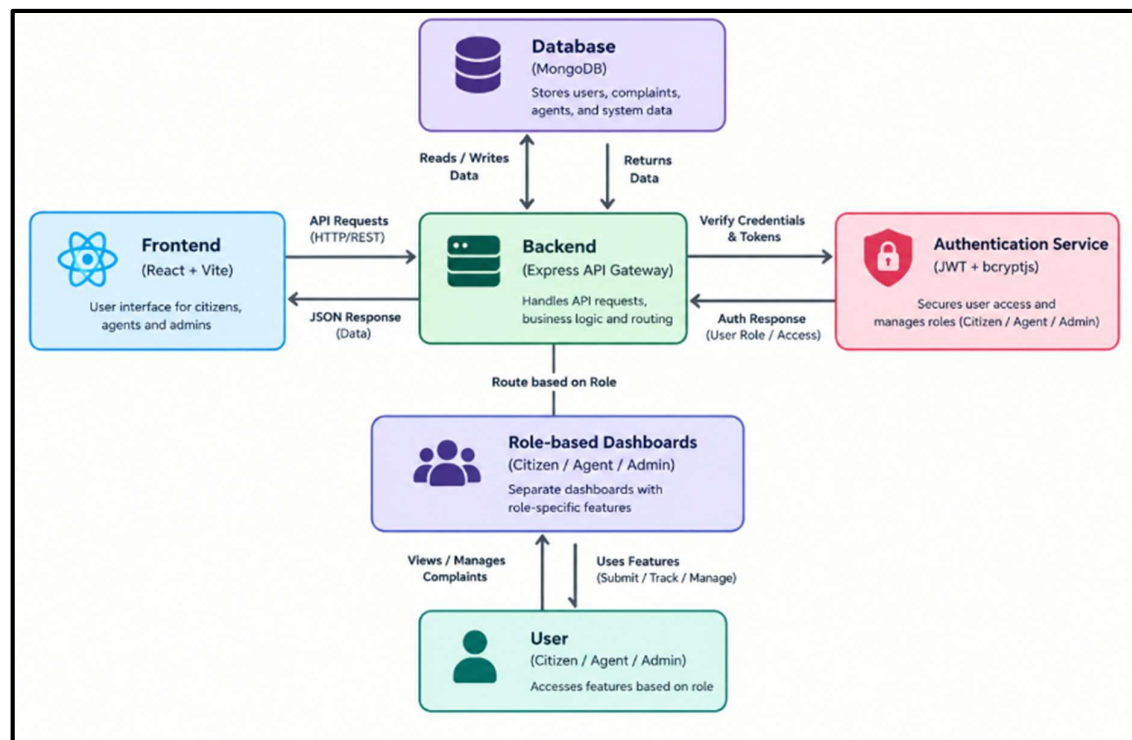
Complaint Management: Citizens submit complaints, while Admins manage and assign complaints to Agents.

Track Complaint: Citizens track their complaint status, while Agents and Admins monitor and update complaint progress.

Resolution Update: Agents or Admins update the complaint status to **In Progress**, **Resolved**, or **Rejected**.

End: The user logs out or exits the **CivicCare** application.

MVC PATTERN:



The **CivicCare** application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for **MongoDB**.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns:**

Each layer has a clearly defined responsibility, improving readability and maintainability.

- **Scalability:**

New features can be added easily by creating new routes, controllers, and models.

- **Reusability:**

Logic in controllers and models can be reused across multiple parts of the application.

- **Testing:**

Each layer can be tested independently, especially the controllers and models.

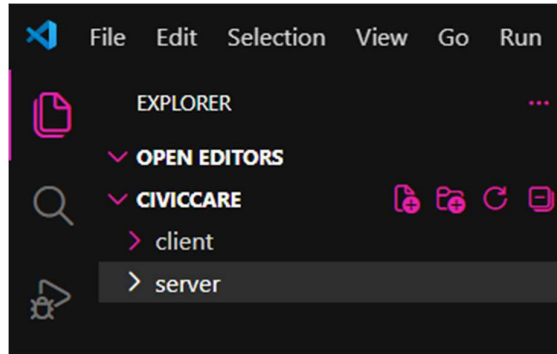
- **Collaboration-Friendly:**

Multiple developers can work simultaneously on different layers without conflict.

Project Setup And Configuration.

- **Creating project folder**

1. Create a new folder with your **CivicCare** project.
2. Inside that folder create two new folders.
3. Name one as **Client**.
4. Name another one as **Server**.
5. Now open that folder in **VS Code**.



- **Client setup (installing React app)**

- Open the **Client** folder in the terminal of VS Code.

- `npm create vite@latest . -- --template react`

- Select **React** framework from the given options.

- Select a framework:

- | React

- Select **JavaScript** variant from the given options.

- Select a variant:

- | JavaScript

- Now let's navigate to the **Client** folder by giving the following command.

- `cd client`

- To install all the packages, run the following command.

- `npm install`

- To start the React server, type the following command.

- `npm run dev`

- **Server setup (npm init)**

- Open **Server** folder in terminal of VS Code.

- `npm init -y`

- Install the required backend packages.

- `npm install express mongoose dotenv cors bcryptjs jsonwebtoken`

- Create files:

- `server.js`

- `.env`

- Create folders:

- `config`

- `models`

- `controllers`

- `middleware`

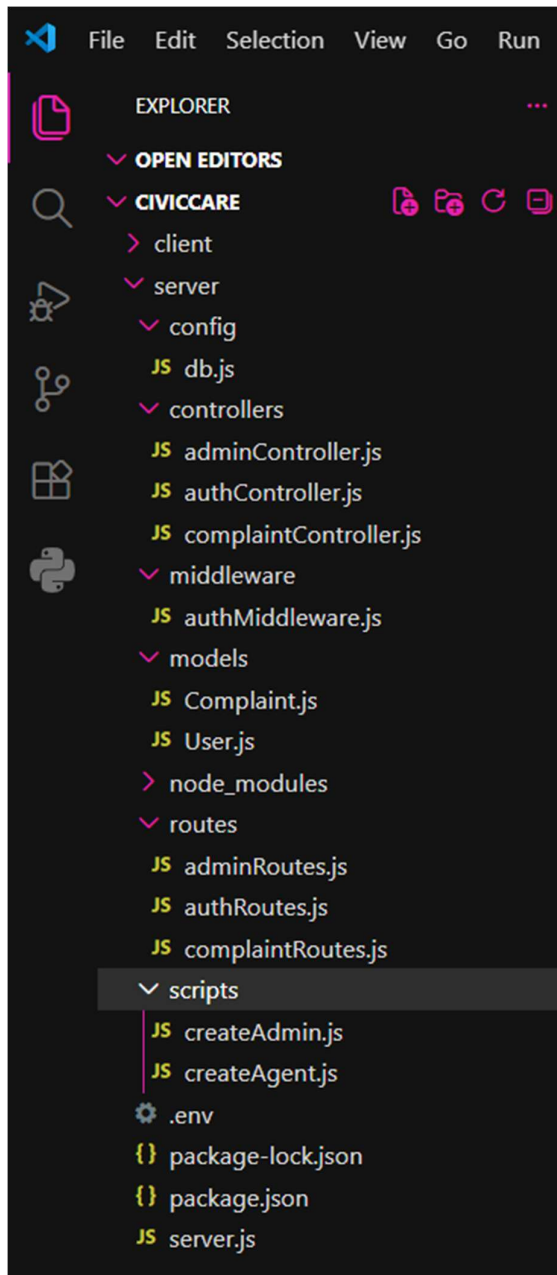
- `routes`

BACKEND DEVELOPMENT:

Backend Reference Video:

<https://drive.google.com/file/d/1B09pPqXxcm9TneR8uafkwUdThcVw49mK/view?usp=sharing>

Backend Structure:



server/

config/

- **db.js** – Sets up and connects the backend to the MongoDB database.

controllers/

- **authController.js** – Handles authentication-related logic such as user registration and login.

- **complaintController.js** – Handles complaint-related logic such as creating, fetching, and updating complaints.

- **adminController.js** – Handles administrator-related operations such as fetching all complaints and updating complaint status.

middleware/

- **authMiddleware.js** – Verifies JWT tokens and protects routes that require authentication. It also provides role-based access for authorized users.

models/

- **User.js** – Defines the Mongoose schema/model for user accounts, including user details and roles such as citizen, agent, and admin.

- **Complaint.js** – Defines the Mongoose schema/model for complaints, including title, description, category, location, status, priority, and associated user.

routes/

- **authRoutes.js** – Defines all authentication-related API routes such as user registration and login.

- **complaintRoutes.js** – Defines complaint-related API routes such as creating complaints, viewing user complaints, and updating complaints.

- **adminRoutes.js** – Defines administrator-related API routes for managing complaints and handling administrative operations.

server.js –

Entry point of the backend; sets up the Express server, middleware, database connection, and API routes.

.env –

Stores environment variables such as the MongoDB connection string, JWT secret, and server configuration.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- **Install Mongoose.**

In your Node.js project directory, run:

```
npm install mongoose
```

- **Install dotenv.**

To manage the MongoDB connection string and other environment variables, run:

```
npm install dotenv
```

- **Create database connection.**

The MongoDB database connection is configured using **Mongoose**. The connection logic is stored in the `config/db.js` file.

- **Create Schemas & Models.**

Mongoose schemas and models are created for the main entities of the CivicCare application, such as **User** and **Complaint**.

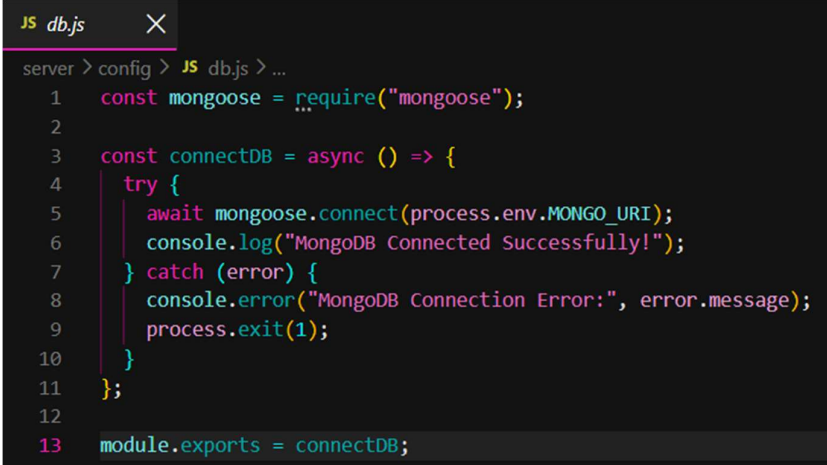
Create Database Connection:

Now, make sure the database is connected before performing any of the actions through the backend. In CivicCare, the MongoDB connection is established using **Mongoose** and the connection string is stored in the `.env` file.

The connection code is placed inside:

```
config/db.js
```

The database connection is then called from `server.js` when the backend server starts. This ensures that the application establishes a connection with MongoDB before handling API requests.



```
JS db.js X
server > config > JS db.js > ...
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect(process.env.MONGO_URI);
6      console.log("MongoDB Connected Successfully!");
7    } catch (error) {
8      console.error("MongoDB Connection Error:", error.message);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
```

Create Schemas:

Firstly, configure the Schemas for the MongoDB database to store the data in such a pattern. Use the data from the ER diagrams to create the schemas. The Schemas for this application look alike to the ones provided below.

User.js – Defines the Mongoose schema/model for user accounts. It stores user information such as name, email, password, and role. The role determines whether the user is a **citizen**, **agent**, or **admin**.

```
JS User.js X
server > models > JS User.js > ...
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: true,
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: true,
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: true,
20     },
21     role: {
22       type: String,
23       enum: ["citizen", "agent", "admin"],
24       default: "citizen",
25     },
26   },
27   {
28     timestamps: true,
29   }
30 );
31 module.exports = mongoose.model("User", userSchema);
```

Complaint.js – Defines the Mongoose schema/model for complaints stored in the database. It stores complaint information such as title, description, category, location, status, priority, and the user associated with the complaint.

```
JS Complaint.js X
server > models > JS Complaint.js > complaintSchema
1  const mongoose = require("mongoose");
2
3  const complaintSchema = new mongoose.Schema(
4    {
5      title: {
6        type: String,
7        required: true,
8        trim: true,
9      },
10     description: {
11       type: String,
12       required: true,
13       trim: true,
14     },
15     category: {
16       type: String,
17       required: true,
18       enum: [
19         "Road",
20         "Water",
21         "Electricity",
22         "Garbage",
23         "Street Light",
24         "Other",
25       ],
26     },
27     location: {
28       type: String,
29       required: true,
30       trim: true,
31     },
```

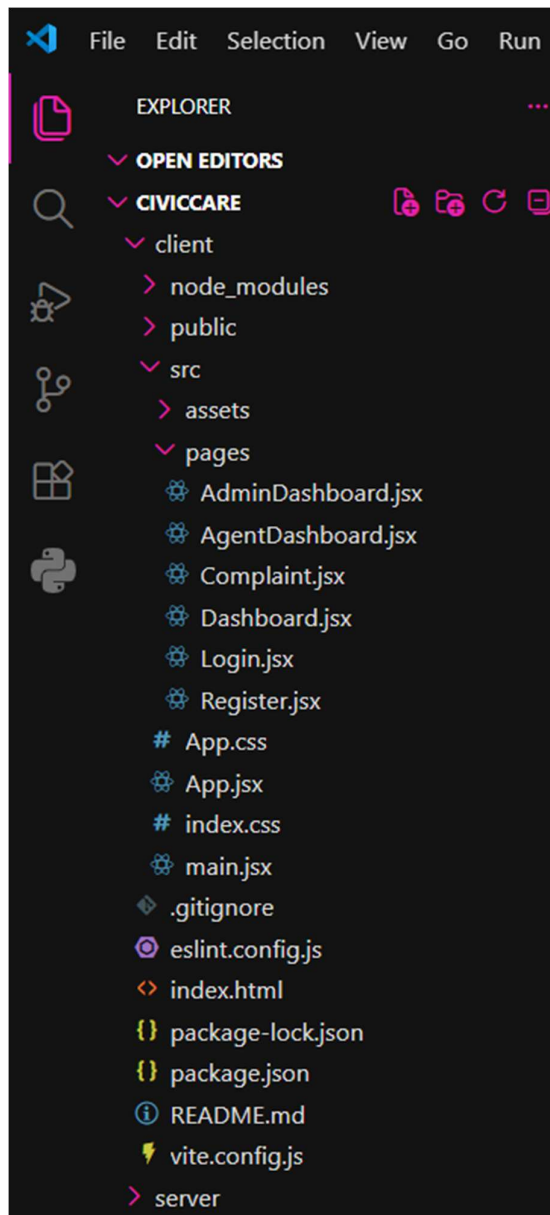
```
32     status: {
33       type: String,
34       enum: ["Pending", "In Progress", "Resolved", "Rejected"],
35       default: "Pending",
36     },
37   },
38
39   priority: {
40     type: String,
41     enum: ["Low", "Medium", "High"],
42     default: "Medium",
43   },
44
45   user: {
46     type: mongoose.Schema.Types.ObjectId,
47     ref: "User",
48     required: true,
49   },
50
51   assignedAgent: {
52     type: mongoose.Schema.Types.ObjectId,
53     ref: "User",
54     default: null,
55   },
56 },
57 {
58   timestamps: true,
59 }
60 );
61
62 module.exports = mongoose.model("Complaint", complaintSchema);
```

FRONT-END DEVELOPMENT:

Frontend Reference Video:

<https://drive.google.com/file/d/12oZrhHO7vcV3FyrSAmW2xGByletT-Pvo/view?usp=sharing>

Frontend Structure:



client/

src/

assets/

Stores images, icons, and other static frontend resources used in the CivicCare application.

pages/

Contains the main pages and dashboards of the CivicCare application.

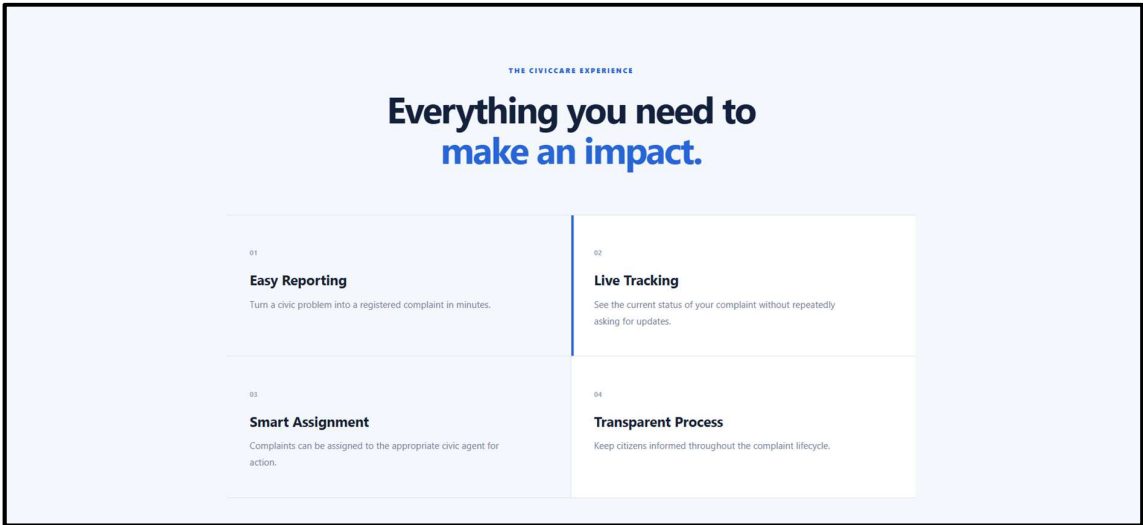
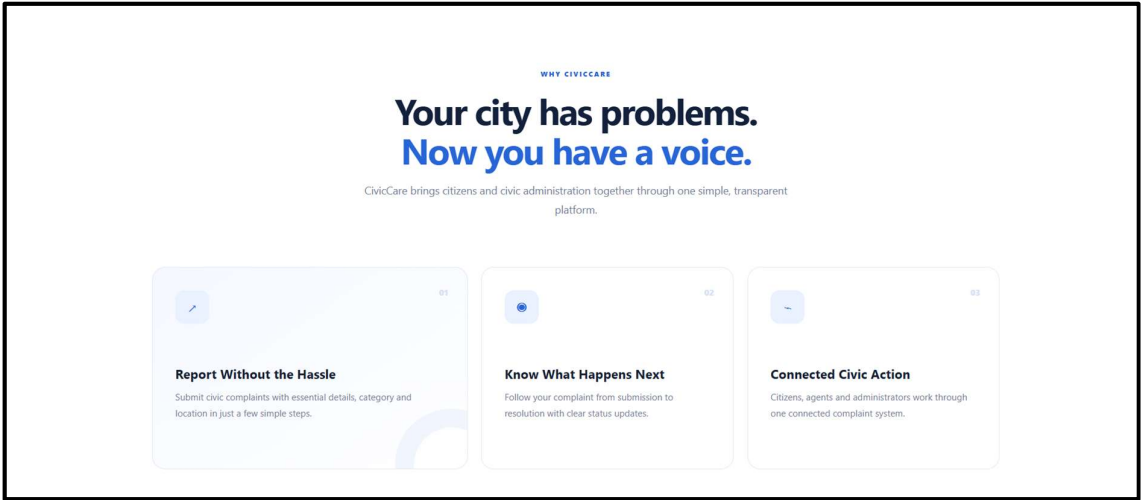
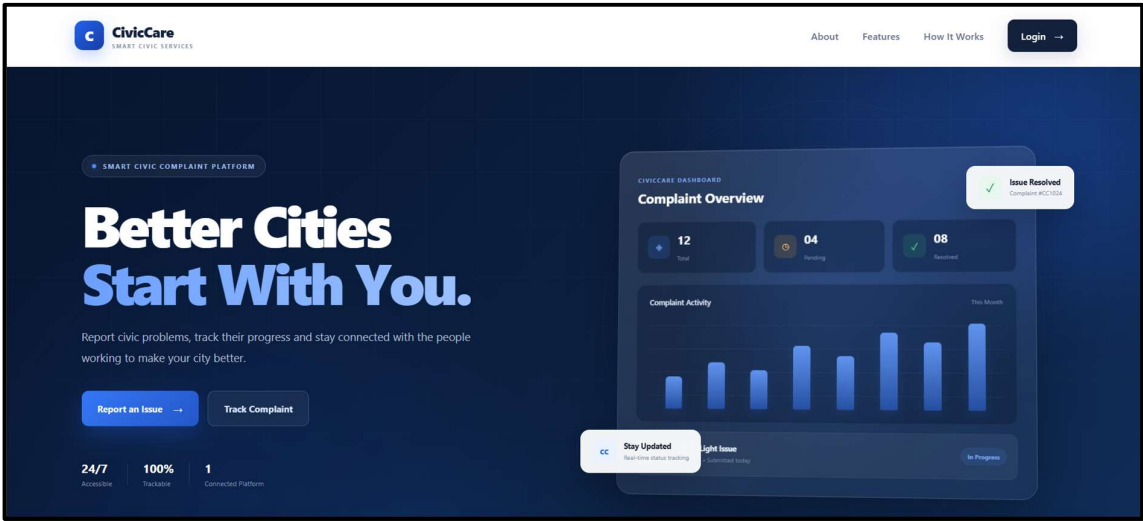
- **Login.jsx** – Provides the login form and handles user authentication.
- **Register.jsx** – Provides the registration form for creating a new user account.
- **CitizenDashboard.jsx** – Provides the citizen interface for registering complaints and viewing complaint details.
- **AdminDashboard.jsx** – Provides the administrator interface for viewing, managing, and assigning complaints.
- **AgentDashboard.jsx** – Provides the agent interface for viewing assigned complaints and updating complaint status.

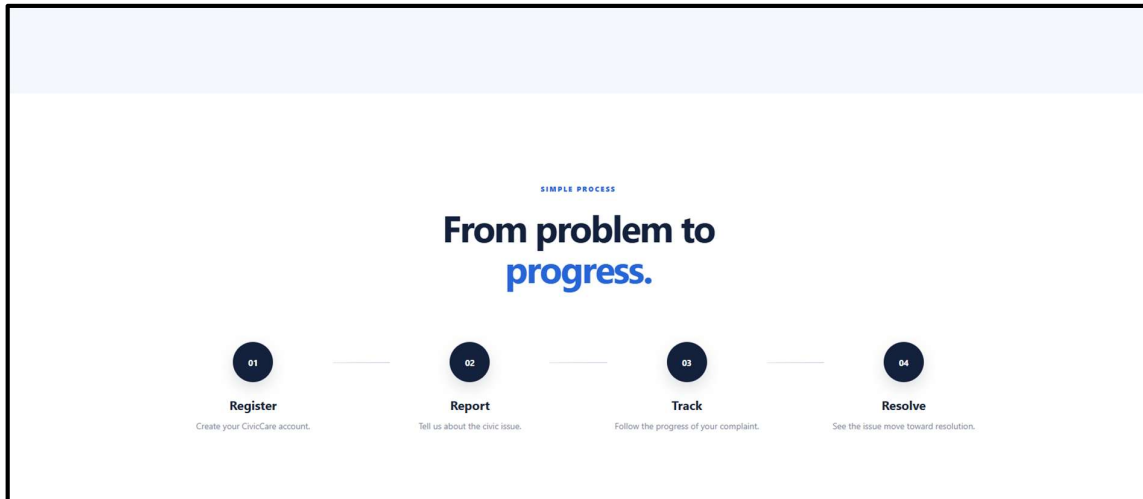
Root Files/

- **App.css** – Contains the main styling for the CivicCare application.
- **App.jsx** – Main component that manages application routes and displays the appropriate pages based on the user's role.
- **index.css** – Contains global CSS styles applied throughout the React application.
- **main.jsx** – Entry point that renders the App component into the DOM.
- **.env** – Stores frontend environment variables such as the backend API URL.
- **.gitignore** – Specifies files and folders that should be ignored by Git.
- **package.json** – Contains project metadata, scripts, and dependencies required for the React application.

Output ScreenShots:

Home Page:





Register Page:

CivicCare

Create Account

Register to report and track civic complaints.

Full Name

Enter your name

Email Address

Enter your email

Password

Create a password

Create Account

Already have an account? [Login here](#)

[← Back to Home](#)

Login Page:

CivicCare

Welcome Back

Login to manage and track your complaints.

Email Address

Enter your email

Password

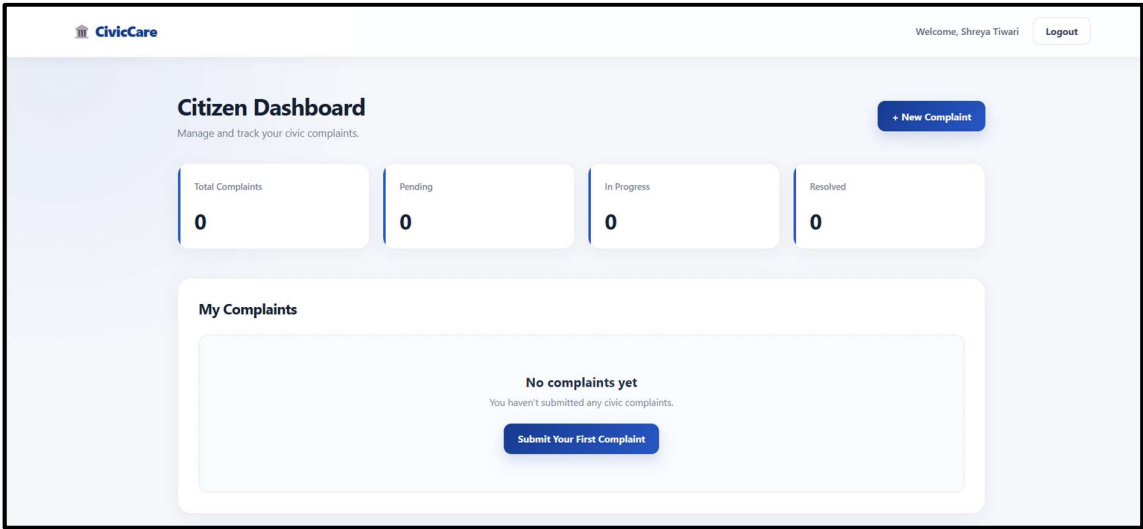
Enter your password

Login

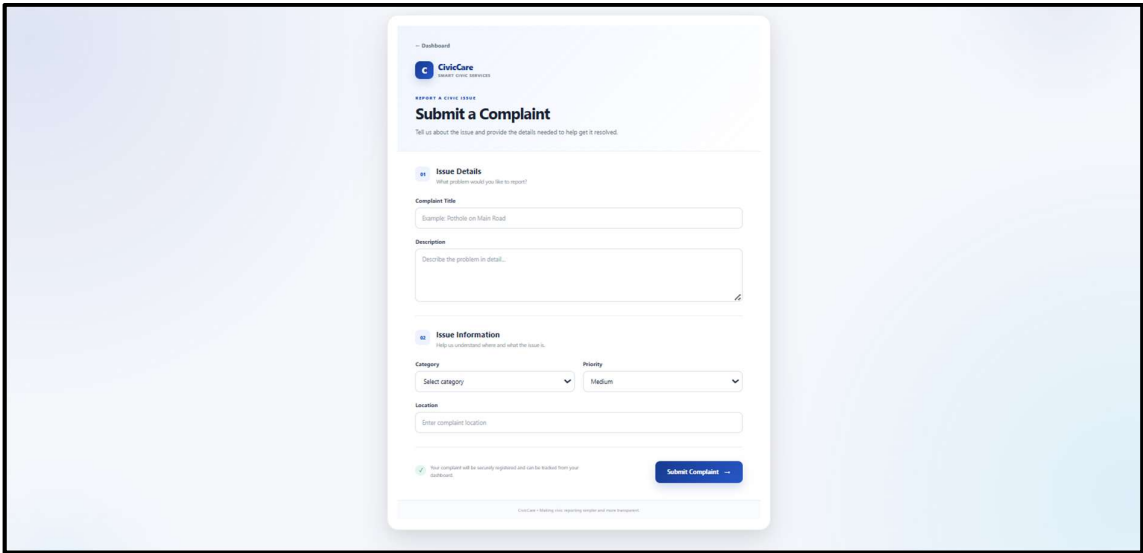
Don't have an account? [Create an account](#)

[← Back to Home](#)

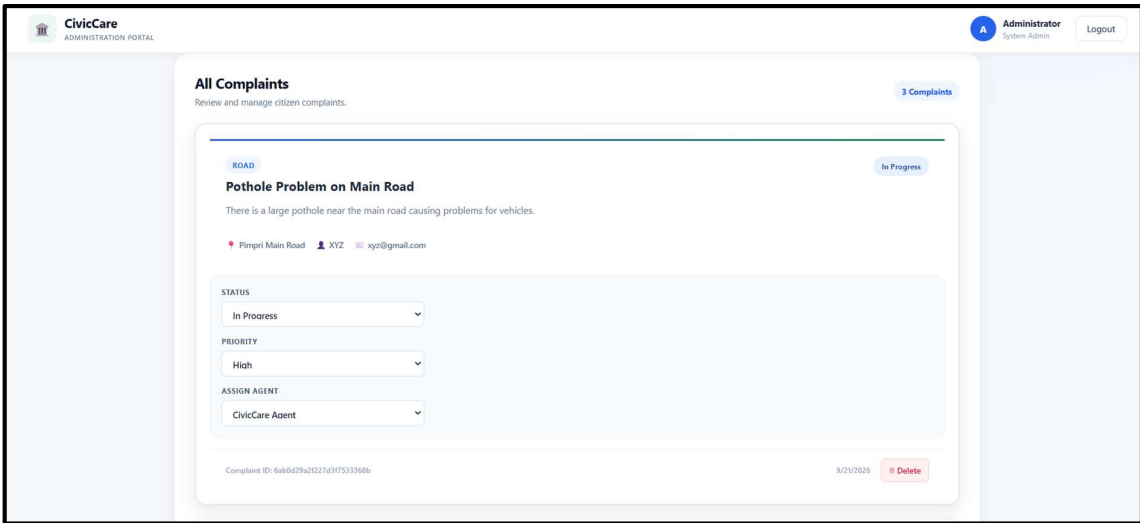
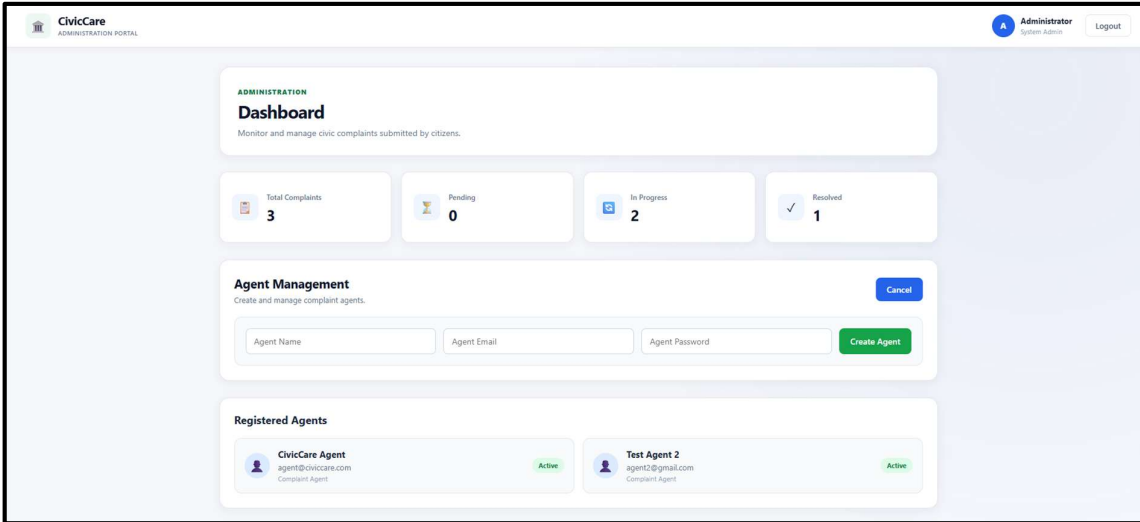
Citizen Dashboard:



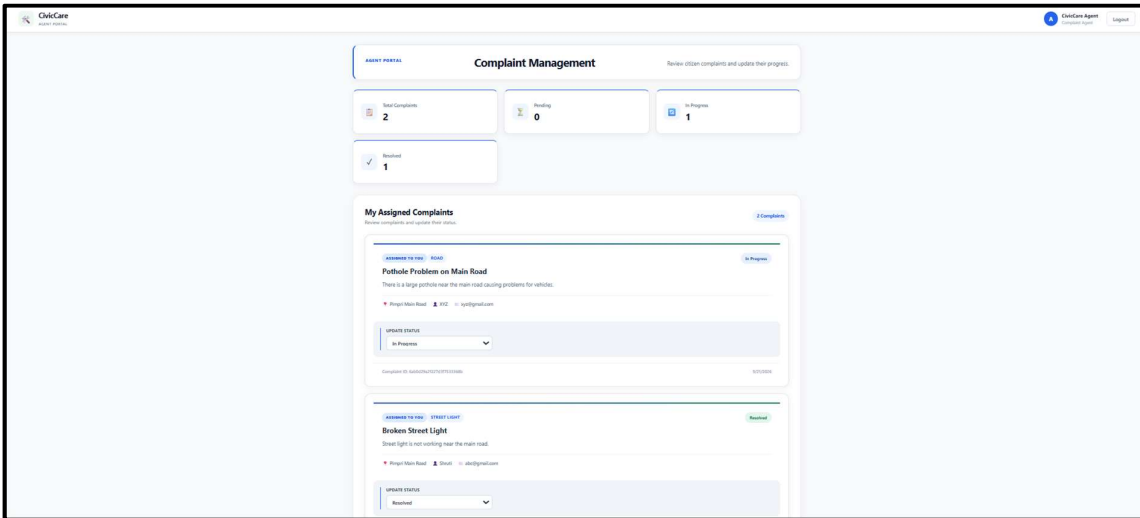
New Complaint:



Admin Dashboard:



Agent Dashboard:



Demo Video Link:

<https://drive.google.com/file/d/1yP0STeRhY61ixshYJvUKohV68k-hE8aA/view?usp=sharing>

Code Drive Link:

<https://github.com/Shruti-Tiwari1709/CivicCare>